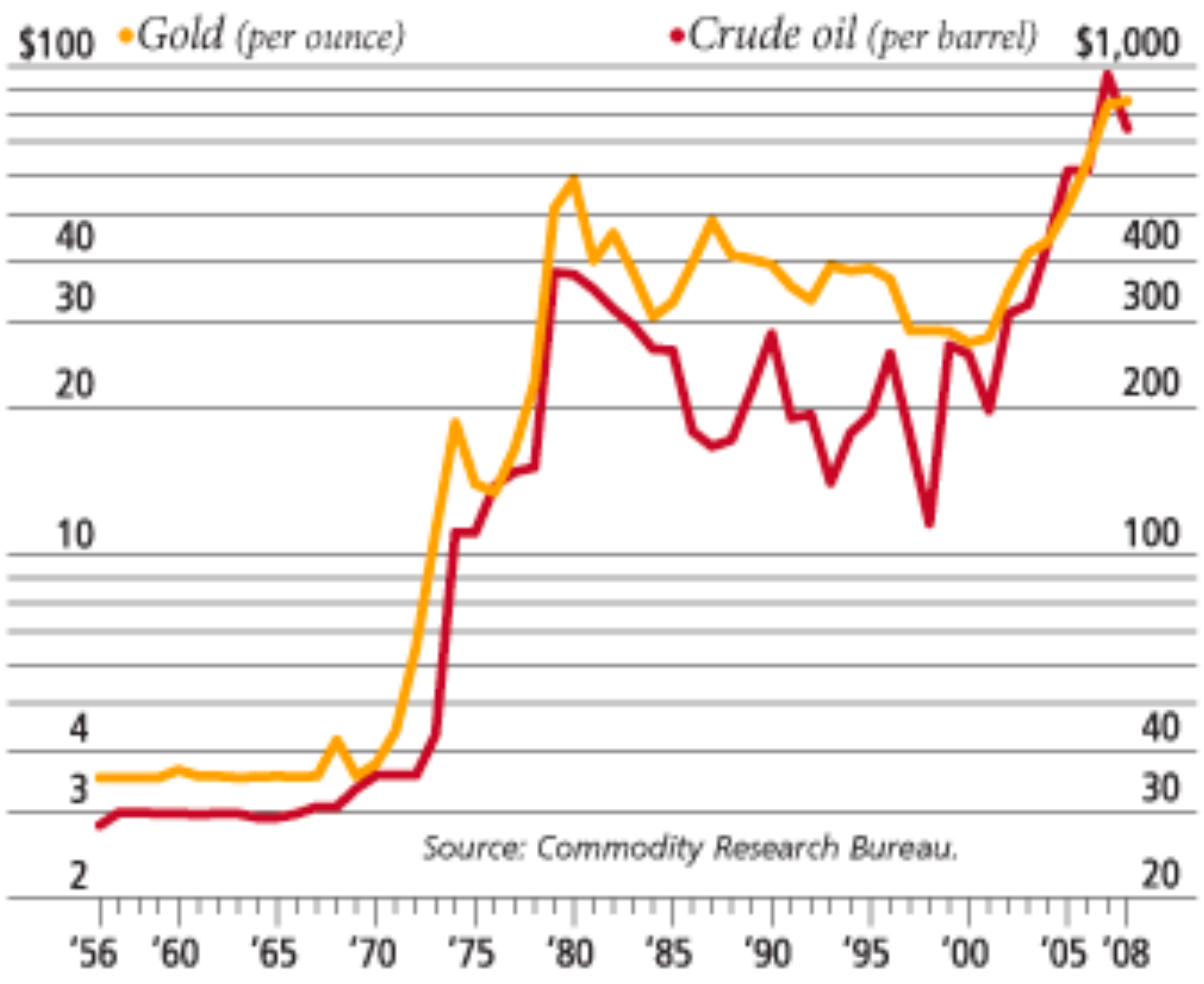
DATA CLEANING

- Time/Date
  - Aligning temporal events from different datasets problematic:
    - Coordinated Universal Time (UTC), GMT, Unix time.
    - Algorithms for converting Gregorian calendar dates to others.

DATA CLEANING

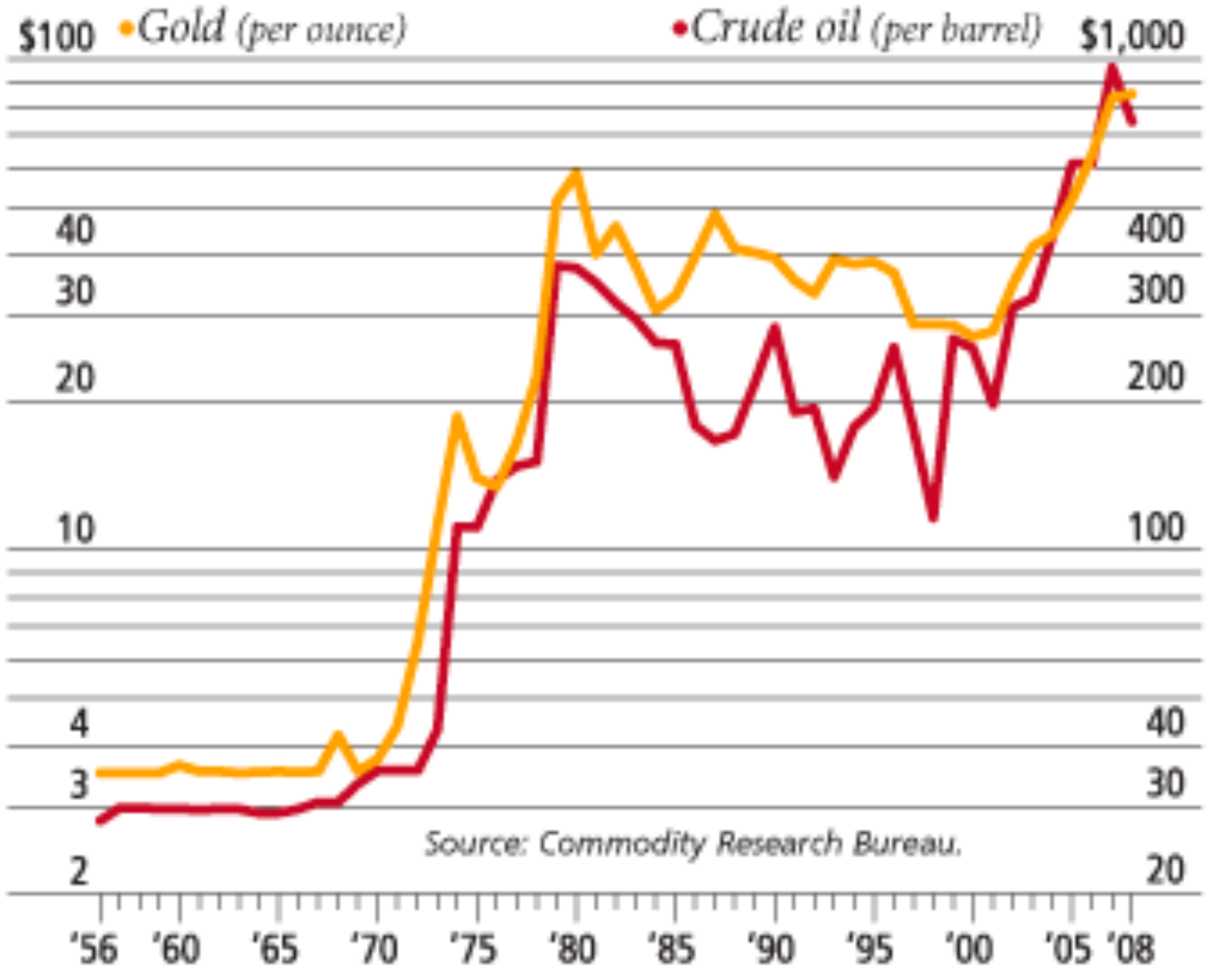
- Financial data
  - Must be careful when units of measurement can change with time
  - Gold/oil prices really correlated?



from forbes.com

DATA CLEANING

- Financial data
  - Must be careful when units of measurement can change with time
  - Gold/oil prices really correlated?



from forbes.com

- Both measured in dollars: Value of dollar changes with time! Time series of prices of any pair of items correlated!

- Solution: Use returns $$r_i = \frac{p_{i+1} - p_i}{p_i}$$

DATA CLEANING

- Dealing with missing values
  - Year of death of a living person?
  - Survey question left blank?

DATA CLEANING

- Dealing with missing values
    - Year of death of a living person?
    - Survey question left blank?

Filling in with values such as 0 or -1 is problematic.

$\Rightarrow$ will be misinterpreted as actual data.

DATA CLEANING

- Dealing with missing values
    - Year of death of a living person?
    - Survey question left blank?

Filling in with values such as 0 or -1 is problematic.

$\Rightarrow$ will be misinterpreted as actual data.

Dropping records with missing values is another option

$\Rightarrow$ other data fields of those records might be valuable

Imputation is a good option.

DATA CLEANING

- Imputation methods

  Heuristic-based imputation: death year(living person)=birth year+80

DATA CLEANING

- Imputation methods

  Heuristic-based imputation: death year(living person)=birth year+80

  Mean value (of the column) imputation: leaves mean the same.

DATA CLEANING

- Imputation methods

  Heuristic-based imputation: death year(living person)=birth year+80

  Mean value (of the column) imputation: leaves mean the same.

  Imputation by nearest neighbor: better than mean when there are systematic reasons to explain variance.

DATA CLEANING

- Imputation methods

Heuristic-based imputation: death year(living person)=birth year+80

Mean value (of the column) imputation: leaves mean the same.

Imputation by nearest neighbor: better than mean when there are systematic reasons to explain variance.

Random value (from the same column) imputation: repeatedly selecting random values permits statistical evaluation of the impact of imputation.

DATA CLEANING

- Imputation methods

Heuristic-based imputation: death year(living person)=birth year+80

Mean value (of the column) imputation: leaves mean the same.

Imputation by nearest neighbor: better than mean when there are systematic reasons to explain variance.

Random value (from the same column) imputation: repeatedly selecting random values permits statistical evaluation of the impact of imputation.

Imputation by interpolation: using linear regression to predict missing values works well if few fields are missing per record.

DATA CLEANING

- Outlier Detection

Dinosaur vertebra 1500mm $\Rightarrow$ 188 feet long beast

Next largest is 122 feet.

If tallest person recorded was 8 feet 9 inches,

Wouldn’t you be surprised of a claim of 12 feet person?

DATA CLEANING

- Outlier Detection

Dinosaur vertebra 1500mm ⇒188 feet long beast

Next largest is 122 feet.

If tallest person recorded was 8 feet 9 inches,

Wouldn’t you be surprised of a claim of 12 feet person?

⇒ Detecting outliers in normally distributed data should be easy (No large outliers)

⇒ the dinosaur example was probably measurement error

⇒ Detecting outliers harder with power-law distributions

DATA CLEANING

- Outlier Detection

  How to detect them?

  Clustering: a data point too far from its cluster center

DATA CLEANING

- Outlier Detection

How to detect them?

Clustering: a data point too far from its cluster center

What to do about them?

Fix why you have outliers.

May hint at systematic problems elsewhere.

Deleting outliers prior to fitting?

May give better models

(if outliers are due to measurement error)

May give worse models

(if removed due to not being explained by your simple model)

Importance of Data Visualization

Anscombe's Quartet: 4 data sets with identical statistical properties.

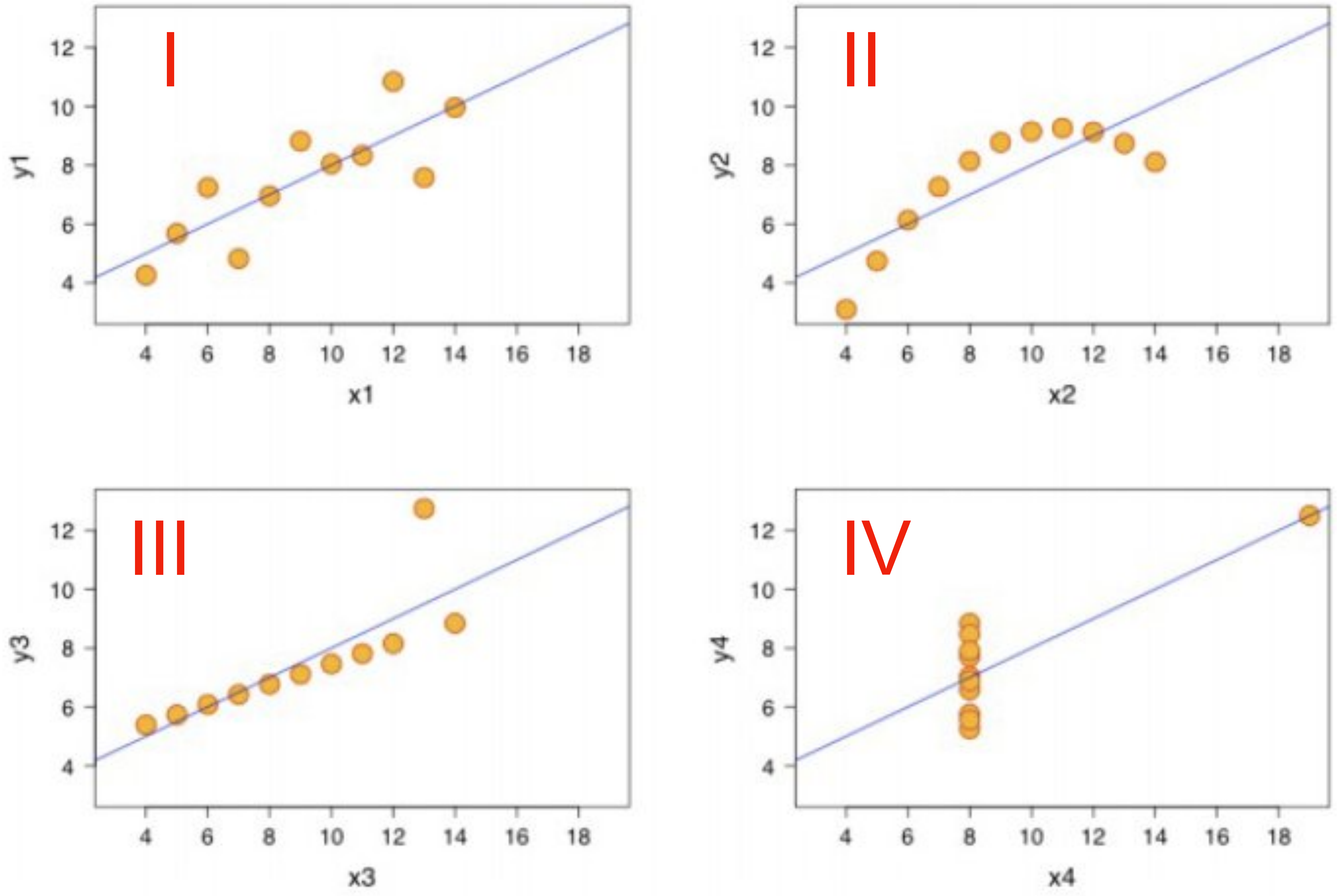
	I		II		III		IV	
	x	y	x	y	x	y	x	y
	10.0	8.04	10.0	9.14	10.0	7.46	8.0	6.58
	8.0	6.95	8.0	8.14	8.0	6.77	8.0	5.76
	13.0	7.58	13.0	8.74	13.0	12.74	8.0	7.71
	9.0	8.81	9.0	8.77	9.0	7.11	8.0	8.84
	11.0	8.33	11.0	9.26	11.0	7.81	8.0	8.47
	14.0	9.96	14.0	8.10	14.0	8.84	8.0	7.04
	6.0	7.24	6.0	6.13	6.0	6.08	8.0	5.25
	4.0	4.26	4.0	3.10	4.0	5.39	19.0	12.50
	12.0	10.84	12.0	9.13	12.0	8.15	8.0	5.56
	7.0	4.82	7.0	7.26	7.0	6.42	8.0	7.91
	5.0	5.68	5.0	4.74	5.0	5.73	8.0	6.89
mean	9.0	7.5	9.0	7.5	9.0	7.5	9.0	7.5
var.	10.0	3.75	10.0	3.75	10.0	3.75	10.0	3.75
corr.	0.816		0.816		0.816		0.816	

DATA VISUALIZATION

Importance of Data Visualization

Anscombe's Quartet: 4 data sets with identical statistical properties.

	I		II		III		IV	
	x	y	x	y	x	y	x	y
	10.0	8.04	10.0	9.14	10.0	7.46	8.0	6.58
	8.0	6.95	8.0	8.14	8.0	6.77	8.0	5.76
	13.0	7.58	13.0	8.74	13.0	12.74	8.0	7.71
	9.0	8.81	9.0	8.77	9.0	7.11	8.0	8.84
	11.0	8.33	11.0	9.26	11.0	7.81	8.0	8.47
	14.0	9.96	14.0	8.10	14.0	8.84	8.0	7.04
	6.0	7.24	6.0	6.13	6.0	6.08	8.0	5.25
	4.0	4.26	4.0	3.10	4.0	5.39	19.0	12.50
	12.0	10.84	12.0	9.13	12.0	8.15	8.0	5.56
	7.0	4.82	7.0	7.26	7.0	6.42	8.0	7.91
	5.0	5.68	5.0	4.74	5.0	5.73	8.0	6.89
mean	9.0	7.5	9.0	7.5	9.0	7.5	9.0	7.5
var.	10.0	3.75	10.0	3.75	10.0	3.75	10.0	3.75
corr.	0.816		0.816		0.816		0.816	



DATA VISUALIZATION

Importance of Data Visualization

- EDA:
    - Identify mistakes in data collection/preprocessing
    - Identify violations of statistical assumptions
    - Observe patterns in the data
    - Construct hypothesis

DATA VISUALIZATION

Importance of Data Visualization

- EDA:
    - Identify mistakes in data collection/preprocessing
    - Identify violations of statistical assumptions
    - Observe patterns in the data
    - Construct hypothesis

- Error Detection:
    - Good idea not to feed unvisualized data into a ML algorithm.
    - Outliers, insufficient cleaning erroneous assumptions etc.

DATA VISUALIZATION

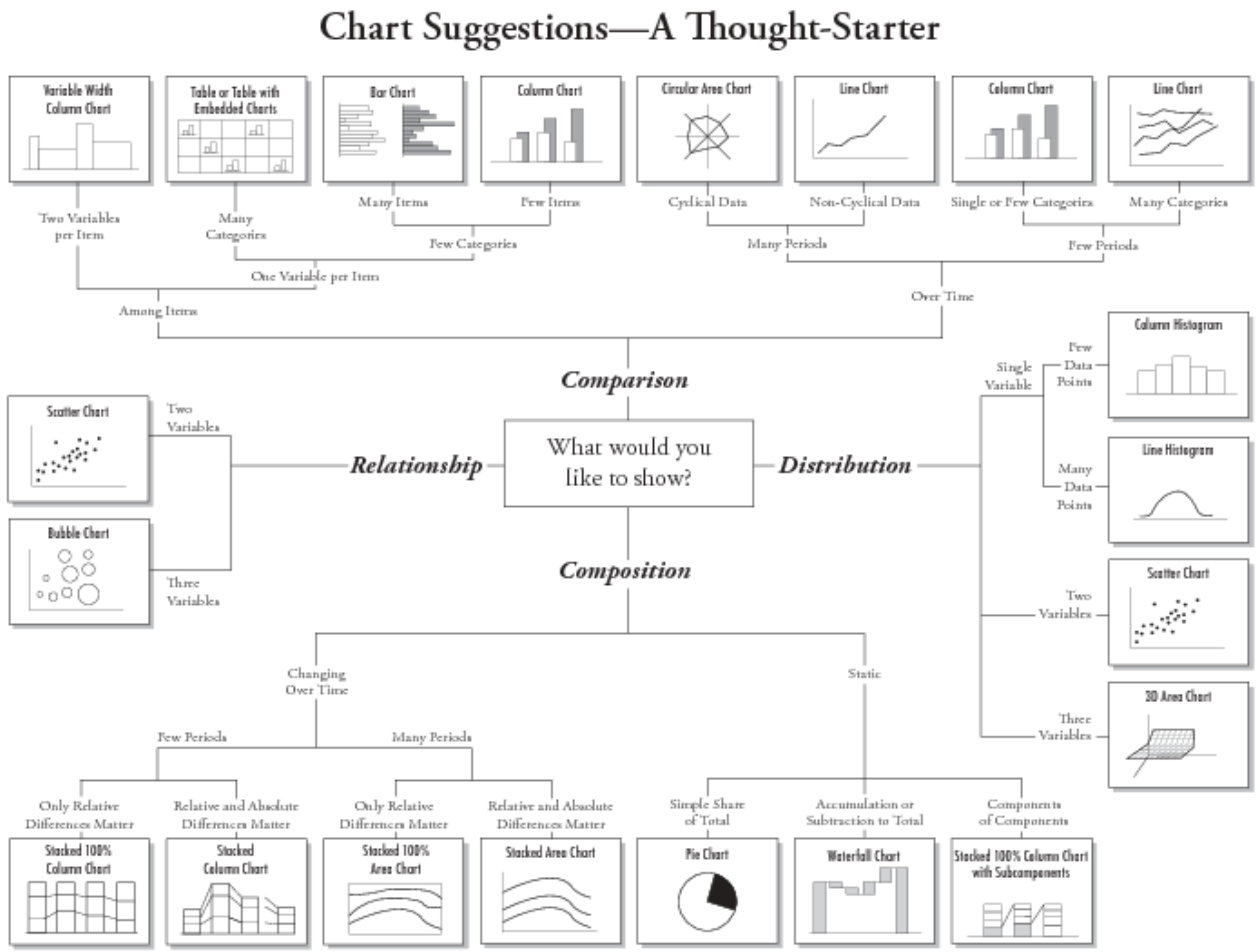
Importance of Data Visualization

- EDA:
  - Identify mistakes in data collection/preprocessing
  - Identify violations of statistical assumptions
  - Observe patterns in the data
  - Construct hypothesis

- Error Detection:
  - Good idea not to feed unvisualized data into a ML algorithm.
  - Outliers, insufficient cleaning, erroneous assumptions etc.

- Communication of findings:
  - A picture is worth 1000 words.

PRACTICE OF DATA VISUALIZATION



PRACTICE OF DATA VISUALIZATION

Visualizing Distributions

Distribution describes:

- The set of values that a variable can possibly take.
- The frequency with which each value occurs.

PRACTICE OF DATA VISUALIZATION

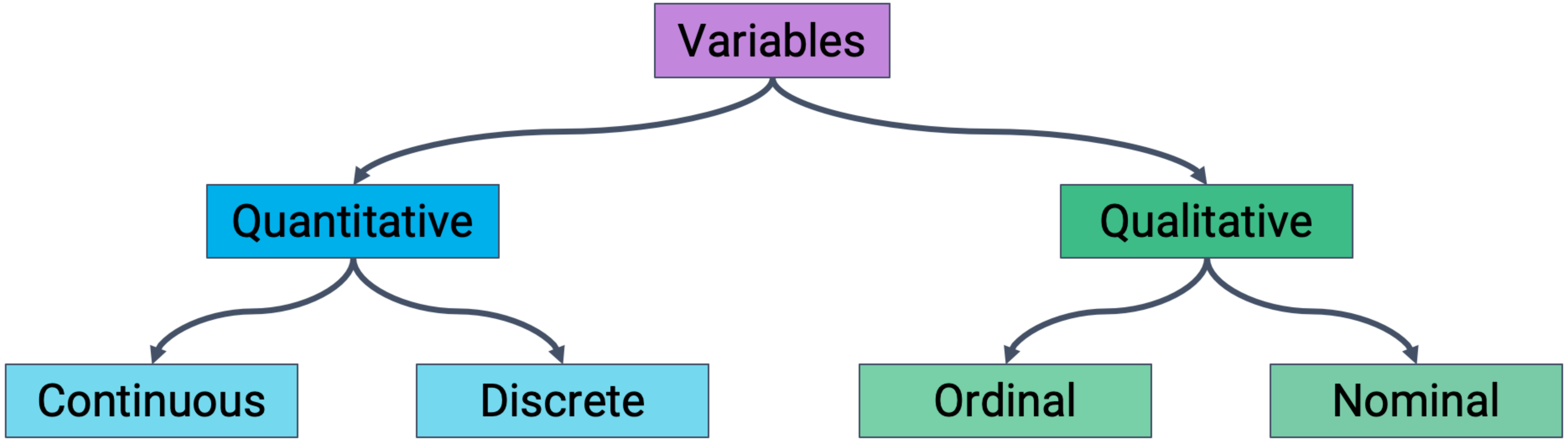
Visualizing Distributions

Distribution describes:

- The set of values that a variable can possibly take.
- The frequency with which each value occurs.

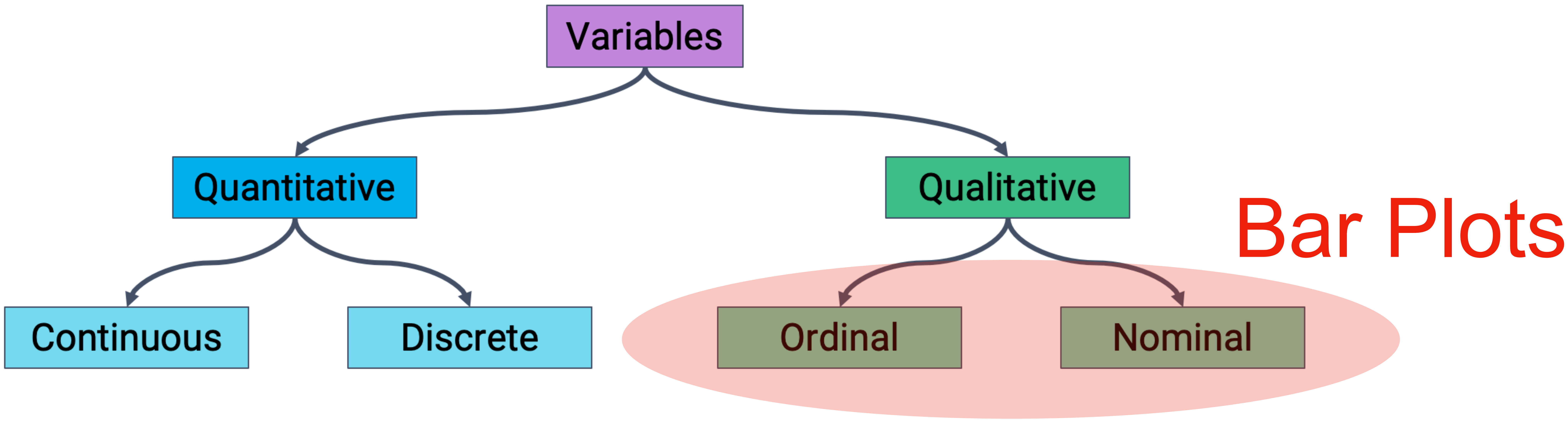
How to visualize a distribution?

- Identify the variables being visualized.
- Select a plot type accordingly.



PRACTICE OF DATA VISUALIZATION

The Python notebook with the codes in the slides can be accessed at:
https://colab.research.google.com/drive/1g3c5Bgry3ynet4_-Tz4CD9jOP4Cg56YE?usp=sharing



PRACTICE OF DATA VISUALIZATION

Bar plots

```
import matplotlib.pyplot as plt
events = ted_main["event"].value_counts().head(5).sort_index()
```

```
TED2009    83
TED2013    77
TED2014    84
TED2015    75
TED2016    77
Name: event, dtype: int64
```

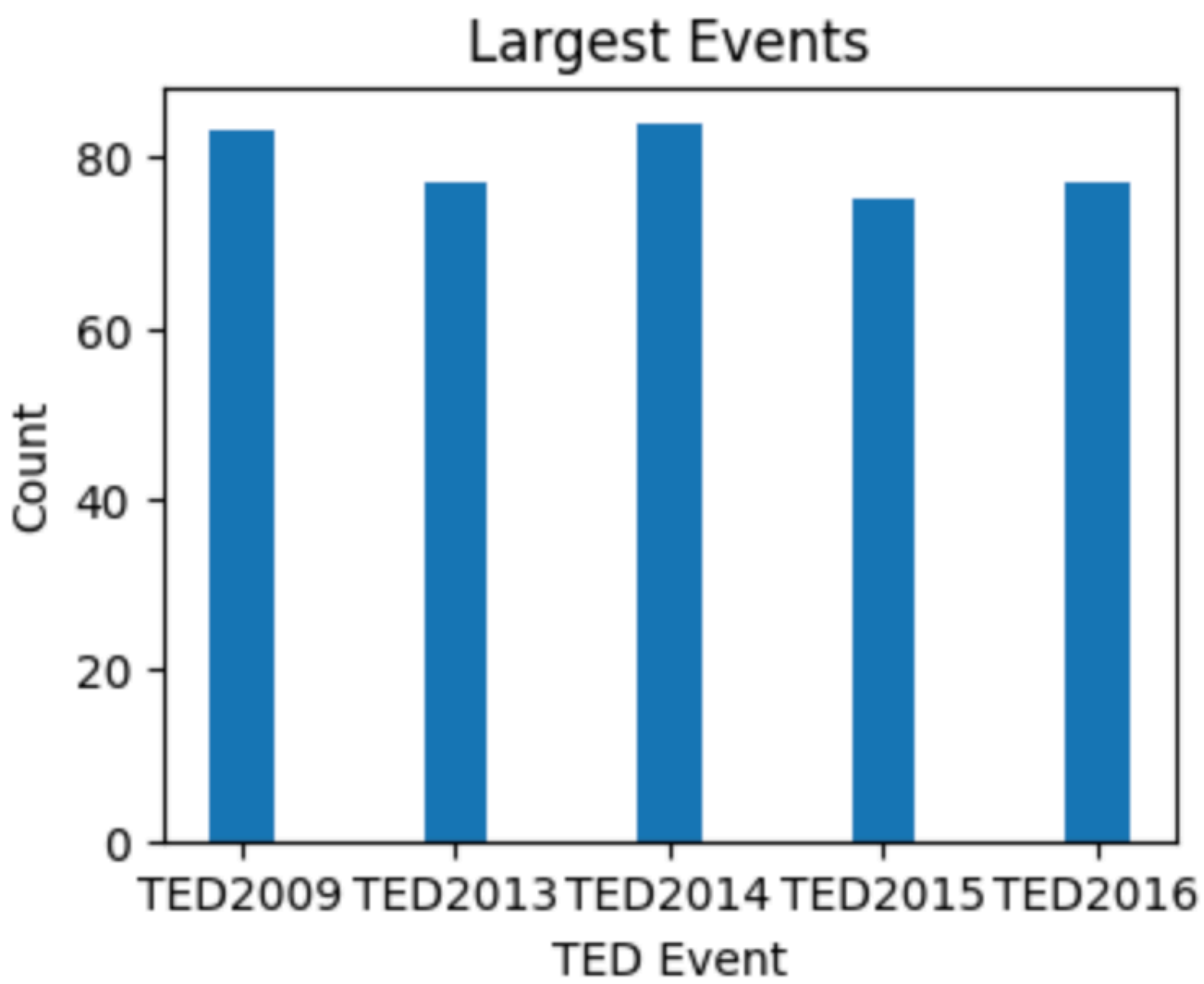
PRACTICE OF DATA VISUALIZATION

Bar plots

```
import matplotlib.pyplot as plt
events = ted_main["event"].value_counts().head(5).sort_index()
```

```
TED2009    83
TED2013    77
TED2014    84
TED2015    75
TED2016    77
Name: event, dtype: int64
```

```
plt.figure(figsize=(4, 3))
plt.bar(events.index, events.values, width=0.3)
plt.xlabel("TED Event")
plt.ylabel("Count")
plt.title("Largest Events")
```



PRACTICE OF DATA VISUALIZATION

Stacked bar plots:

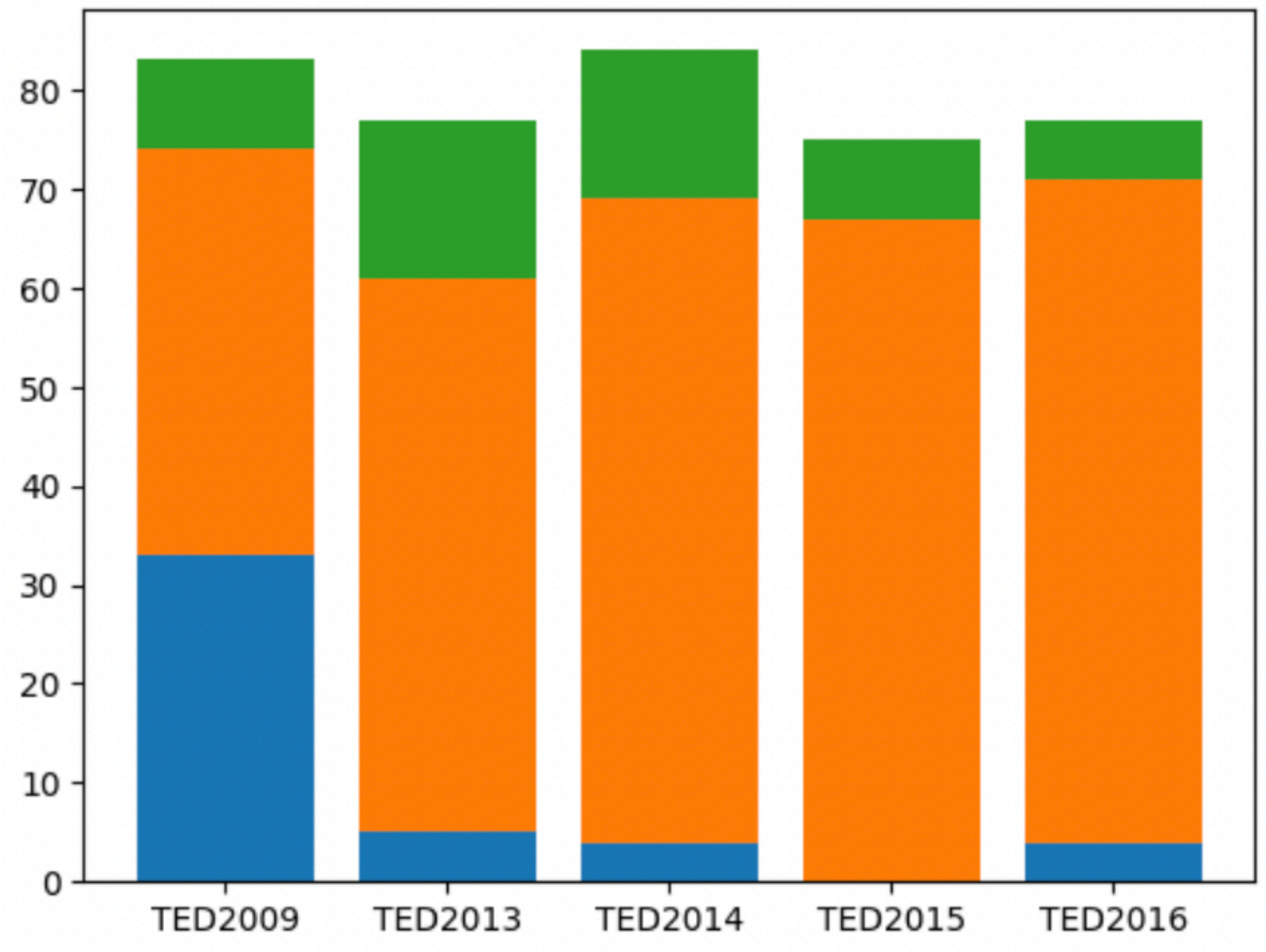
```
p1 = [(x, len(ted_main[(ted_main["event"]==x) &
                        (ted_main["views"]<=800000)]))
      for x in events.index]
p2 = [(x, len(ted_main[(ted_main["event"]==x) &
                        (ted_main["views"]>800000) &
                        (ted_main["views"]<=3000000)]))
      for x in events.index]
p3 = [(x, len(ted_main[(ted_main["event"]==x) &
                        (ted_main["views"]>3000000)]))
      for x in events.index]
```

PRACTICE OF DATA VISUALIZATION

Stacked bar plots:

```
p1 = [(x, len(ted_main[(ted_main["event"]==x) &
                         (ted_main["views"]<=800000)]))
      for x in events.index]
p2 = [(x, len(ted_main[(ted_main["event"]==x) &
                         (ted_main["views"]>800000) &
                         (ted_main["views"]<=3000000)]))
      for x in events.index]
p3 = [(x, len(ted_main[(ted_main["event"]==x) &
                         (ted_main["views"]>3000000)]))
      for x in events.index]
```

```
plt.bar([d[0] for d in p1], [d[1] for d in p1])
plt.bar([d[0] for d in p2], [d[1] for d in p2],
        bottom=[d[1] for d in p1])
plt.bar([d[0] for d in p3], [d[1] for d in p3],
        bottom=np.array([d[1] for d in p1])+np.array([d[1] for d in p2]))
```



PRACTICE OF DATA VISUALIZATION

Ex: Bar plots not suitable for quantitative variables.

```
cd_ratio = ted_main["comments"] / ted_main["duration"]
cd_ratio = cd_ratio.value_counts().head(100)
cd_ratio
```

```
plt.figure(figsize=(8, 3))
plt.bar(cd_ratio.index, cd_ratio.values, width=0.005)
plt.xlabel("C/D Ratios")
plt.ylabel("Count")
plt.title("Comments/Duration Ratio Counts")
```

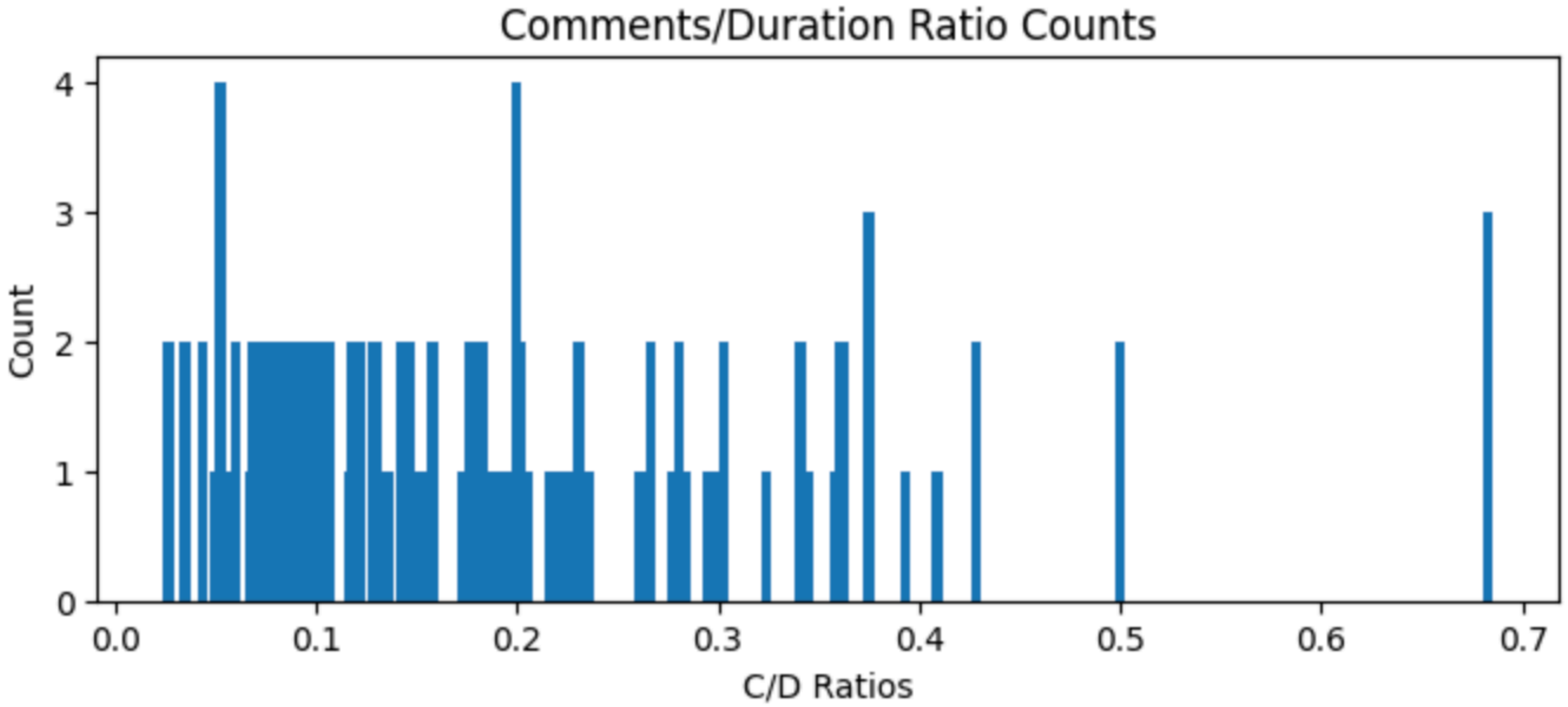
PRACTICE OF DATA VISUALIZATION

Ex: Bar plots not suitable for quantitative variables.

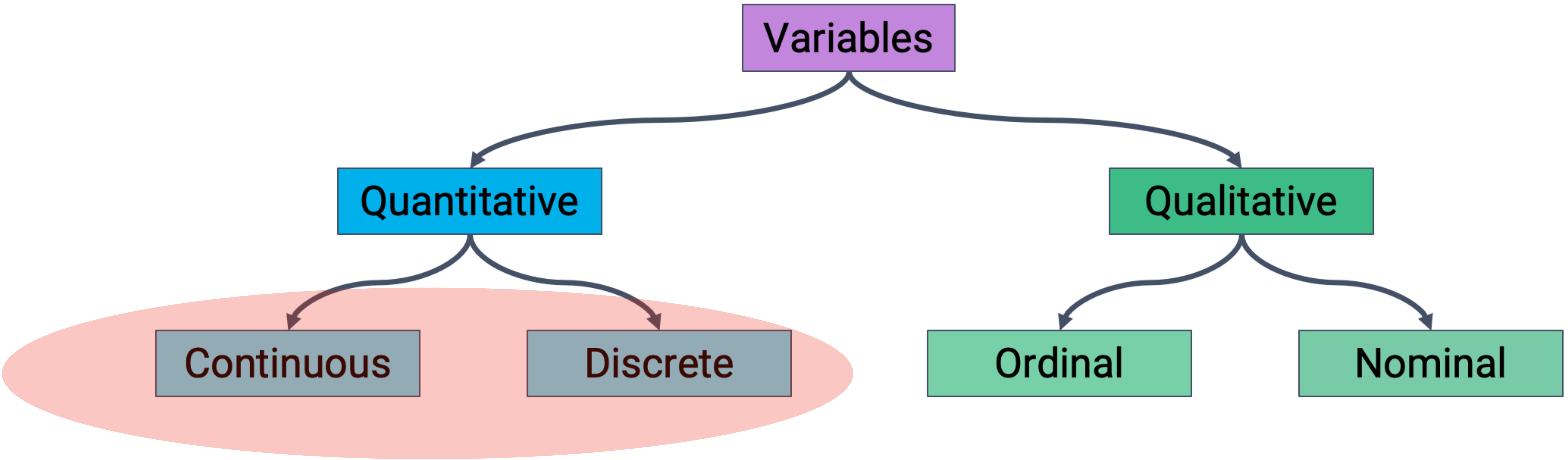
```
cd_ratio = ted_main["comments"] / ted_main["duration"]
cd_ratio = cd_ratio.value_counts().head(100)
cd_ratio
```

```
plt.figure(figsize=(8, 3))
plt.bar(cd_ratio.index, cd_ratio.values, width=0.005)
plt.xlabel("C/D Ratios")
plt.ylabel("Count")
plt.title("Comments/Duration Ratio Counts")
```

Separate bar for each unique value. Too many bars for continuous data or discrete data with many unique values.



PRACTICE OF DATA VISUALIZATION



How to visualize?

PRACTICE OF DATA VISUALIZATION

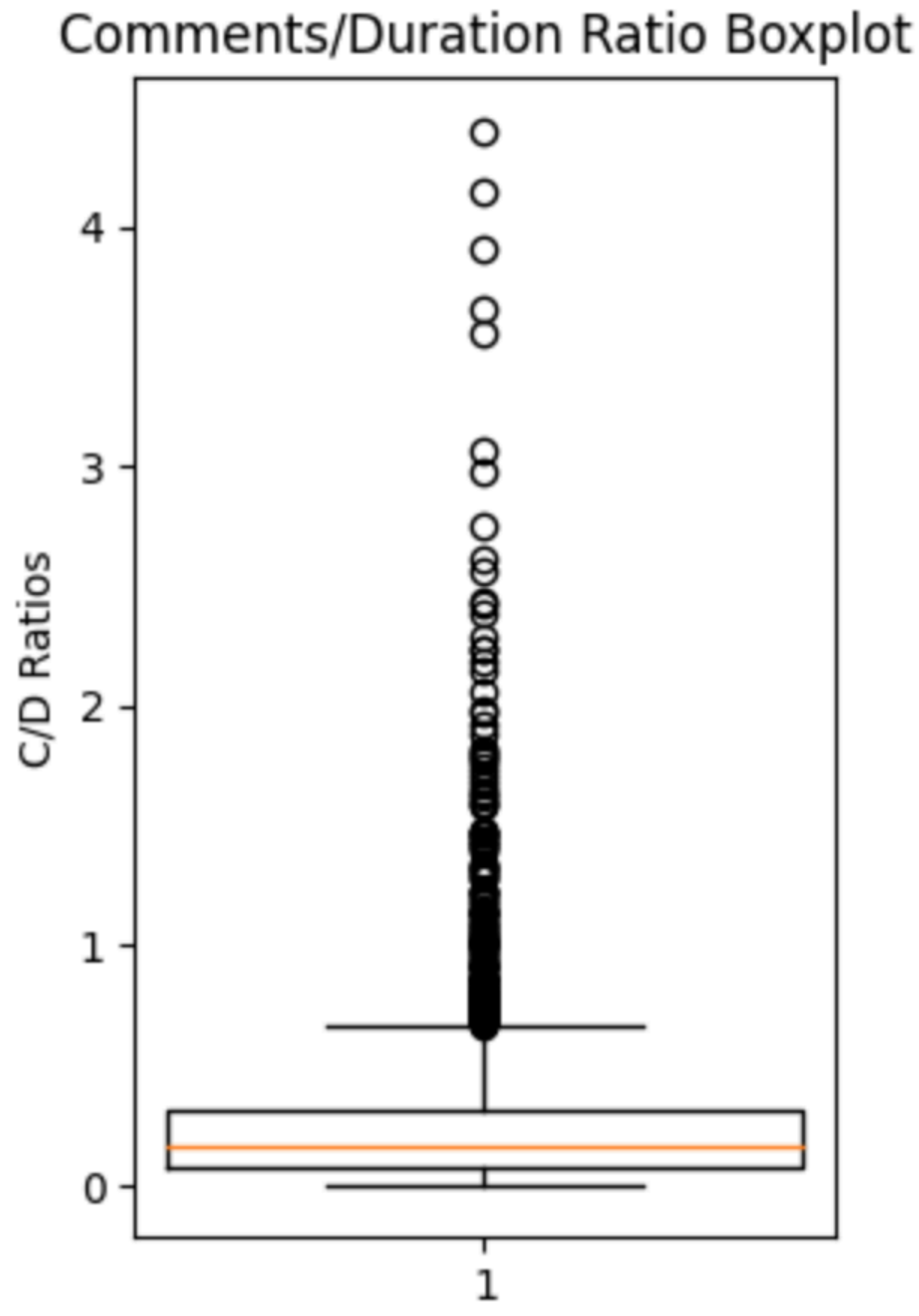
Box Plots: for visualizing a continuous quantitative variable.

```
plt.figure(figsize=(3, 5))
plt.boxplot(ted_main["comments"] / ted_main["duration"], widths=(1.5))
plt.ylabel("C/D Ratios")
plt.title("Comments/Duration Ratio Boxplot")
```

PRACTICE OF DATA VISUALIZATION

Box Plots: for visualizing a continuous quantitative variable.

```
plt.figure(figsize=(3, 5))
plt.boxplot(ted_main["comments"] / ted_main["duration"], widths=(1.5))
plt.ylabel("C/D Ratios")
plt.title("Comments/Duration Ratio Boxplot")
```



PRACTICE OF DATA VISUALIZATION

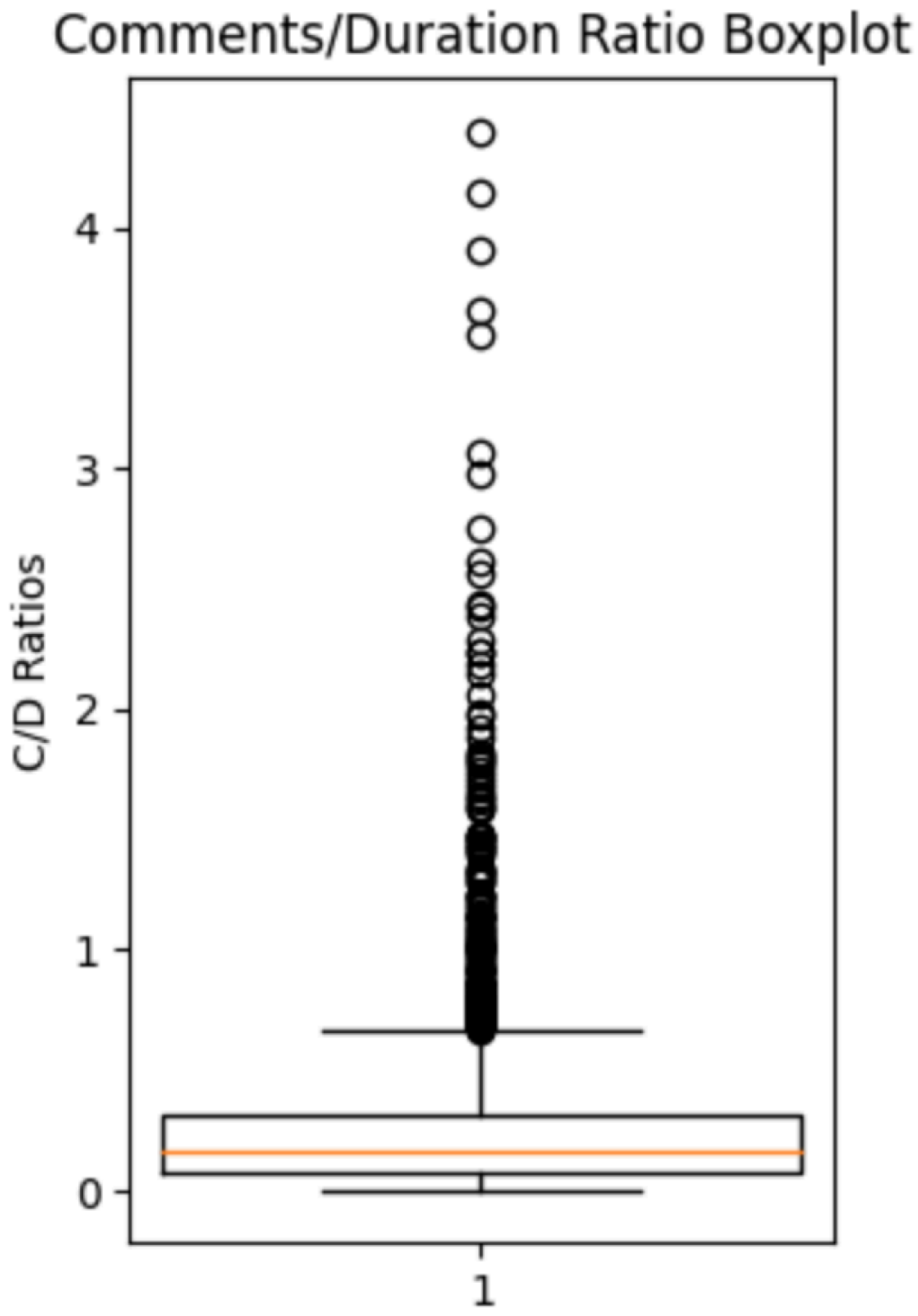
Box Plots: for visualizing a continuous quantitative variable.

Display distribution using quartiles:

- First or lower quartile: 25th percentile.
- Second quartile: 50th percentile (median).
- Third or upper quartile: 75th percentile.

[1^{st}quartile, 3^{rd}quartile]: "middle $50\ \%$" of data

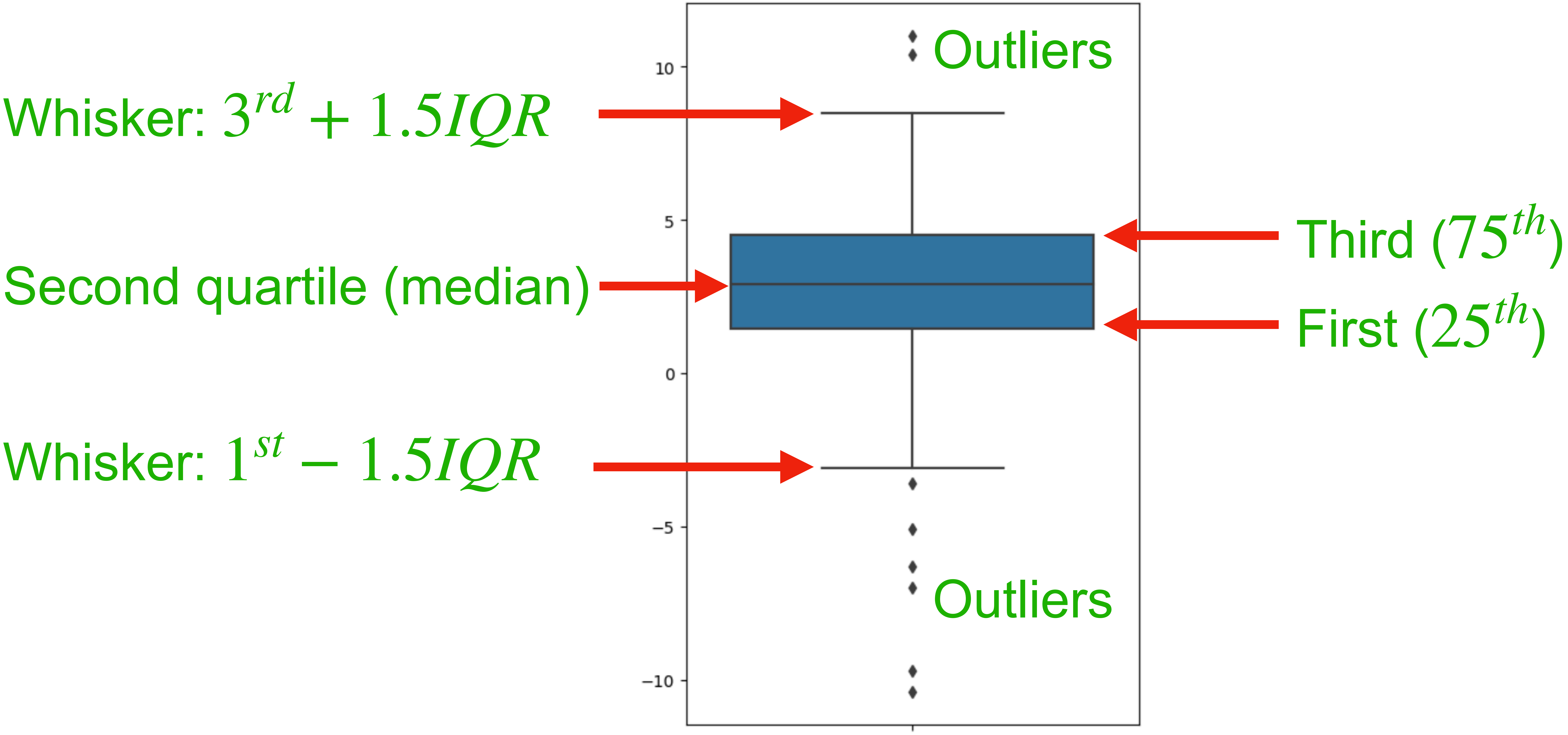
IQR: 3^{rd} quartile $-\, 1^{st}$ quartile



Width of the box encodes no information.

PRACTICE OF DATA VISUALIZATION

Box Plots: for visualizing a continuous quantitative variable.



PRACTICE OF DATA VISUALIZATION

Violin Plots: for visualizing quantitative variable.

This time let's use the seaborn library.

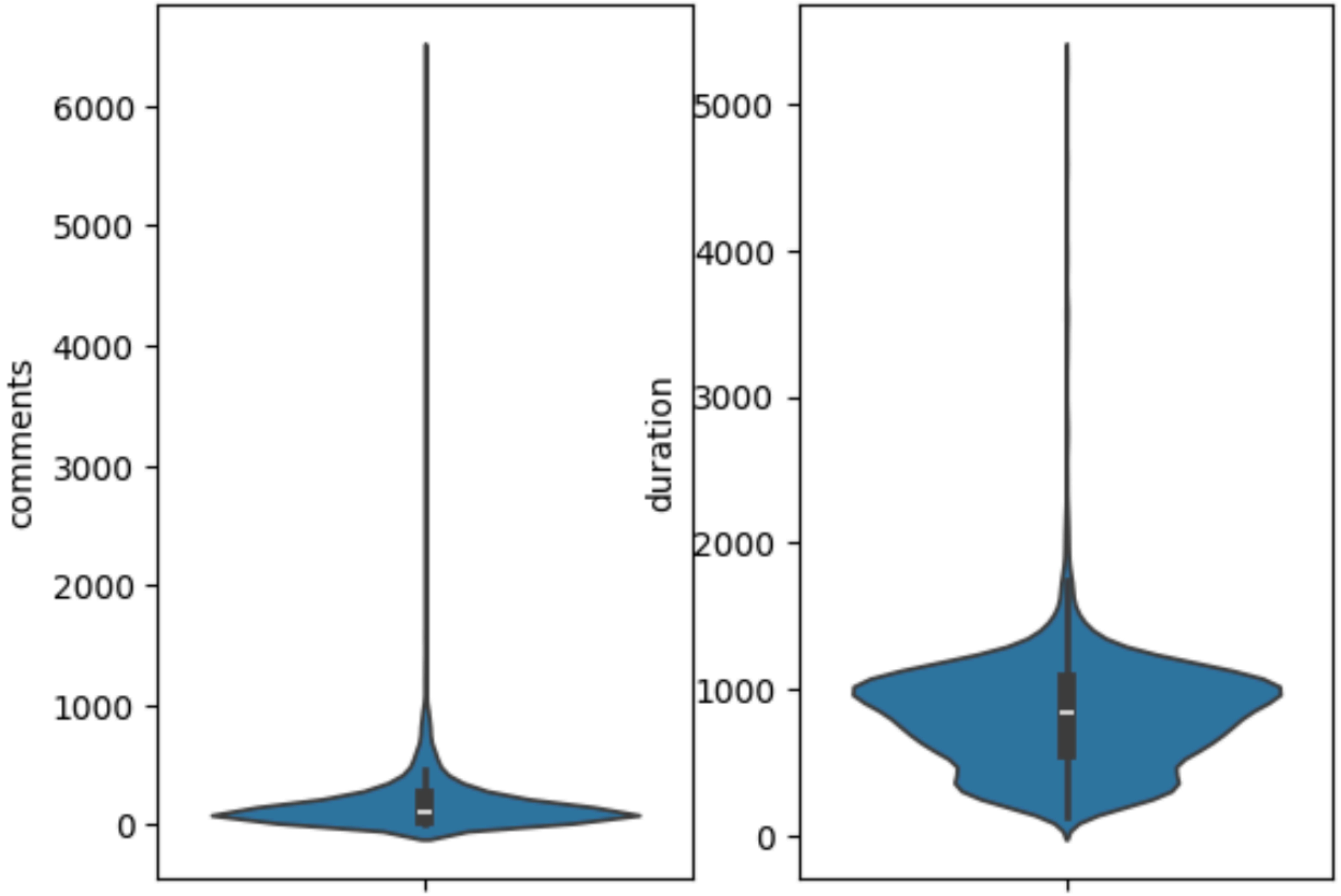
```
import seaborn as sns
fig, (ax1, ax2) = plt.subplots(1, 2)
sns.violinplot(ted_main["comments"], ax=ax1)
sns.violinplot(ted_main["duration"], ax=ax2)
```

PRACTICE OF DATA VISUALIZATION

Violin Plots: for visualizing quantitative variable.

This time let's use the seaborn library.

```
import seaborn as sns
fig, (ax1, ax2) = plt.subplots(1, 2)
sns.violinplot(ted_main["comments"], ax=ax1)
sns.violinplot(ted_main["duration"], ax=ax2)
```



Also shows smoothed density curves: Width has a meaning
Quartiles/whiskers still there.

PRACTICE OF DATA VISUALIZATION

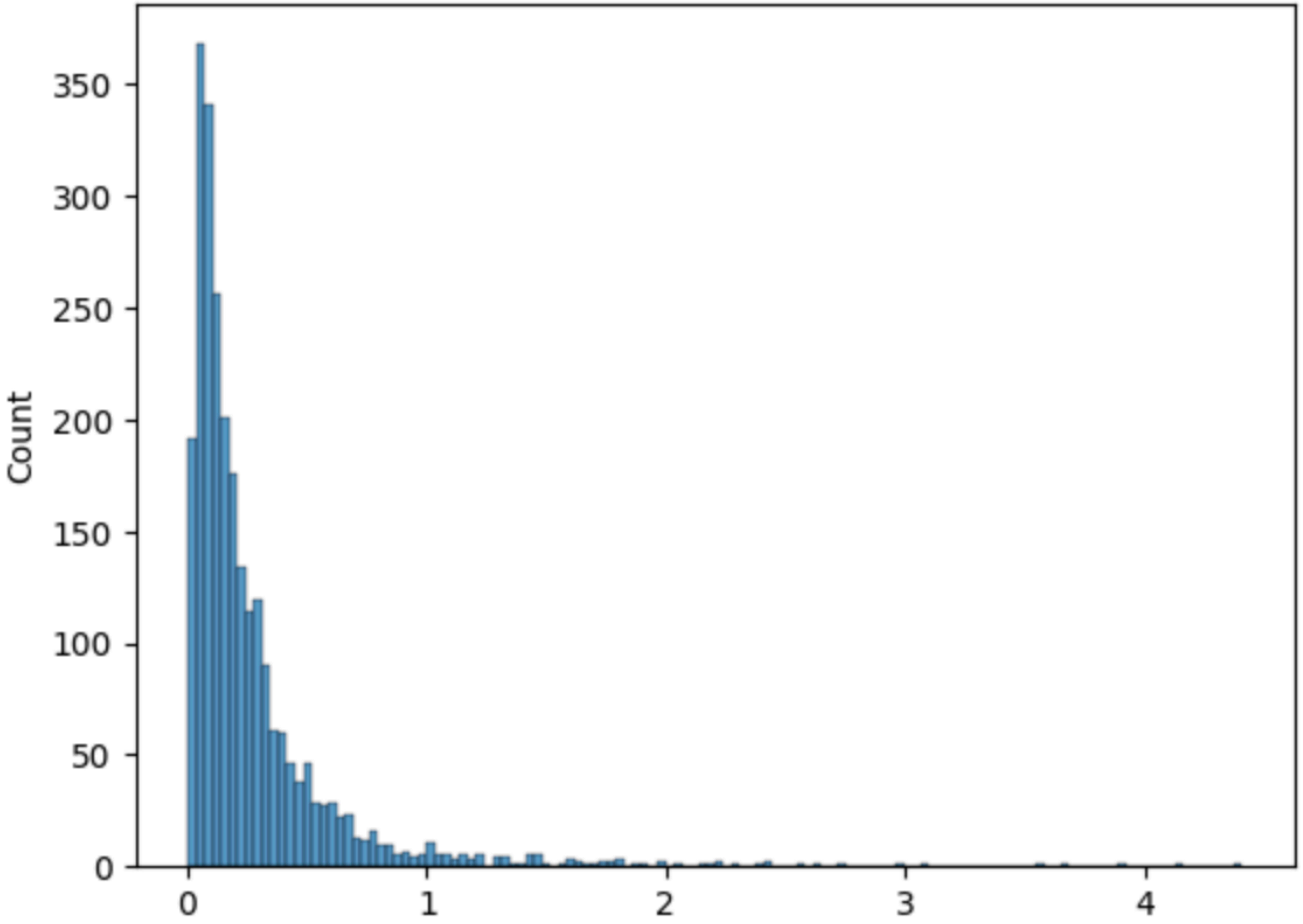
Histogram: collects data points with similar values into bins

```
sns.histplot(ted_main["comments"]/ted_main["duration"])
```

PRACTICE OF DATA VISUALIZATION

Histogram: collects data points with similar values into bins

```
sns.histplot(ted_main["comments"]/ted_main["duration"])
```



Bin area = % of datapoints in it.

Ex: For the 1^{st} bin:

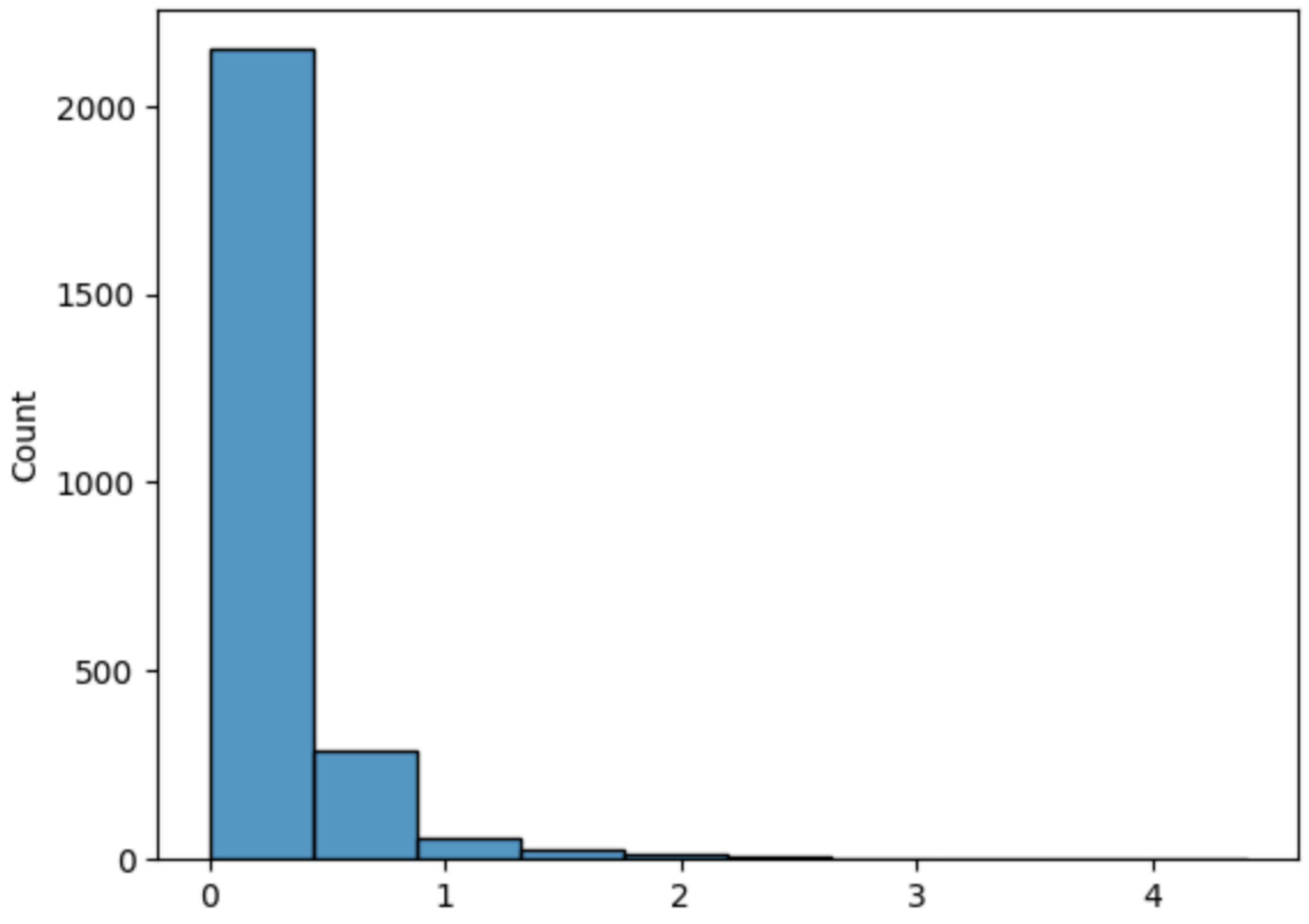
width = 0.033, height=190

$$\Rightarrow 0.033 \times 190 = 6.3\,\%$$

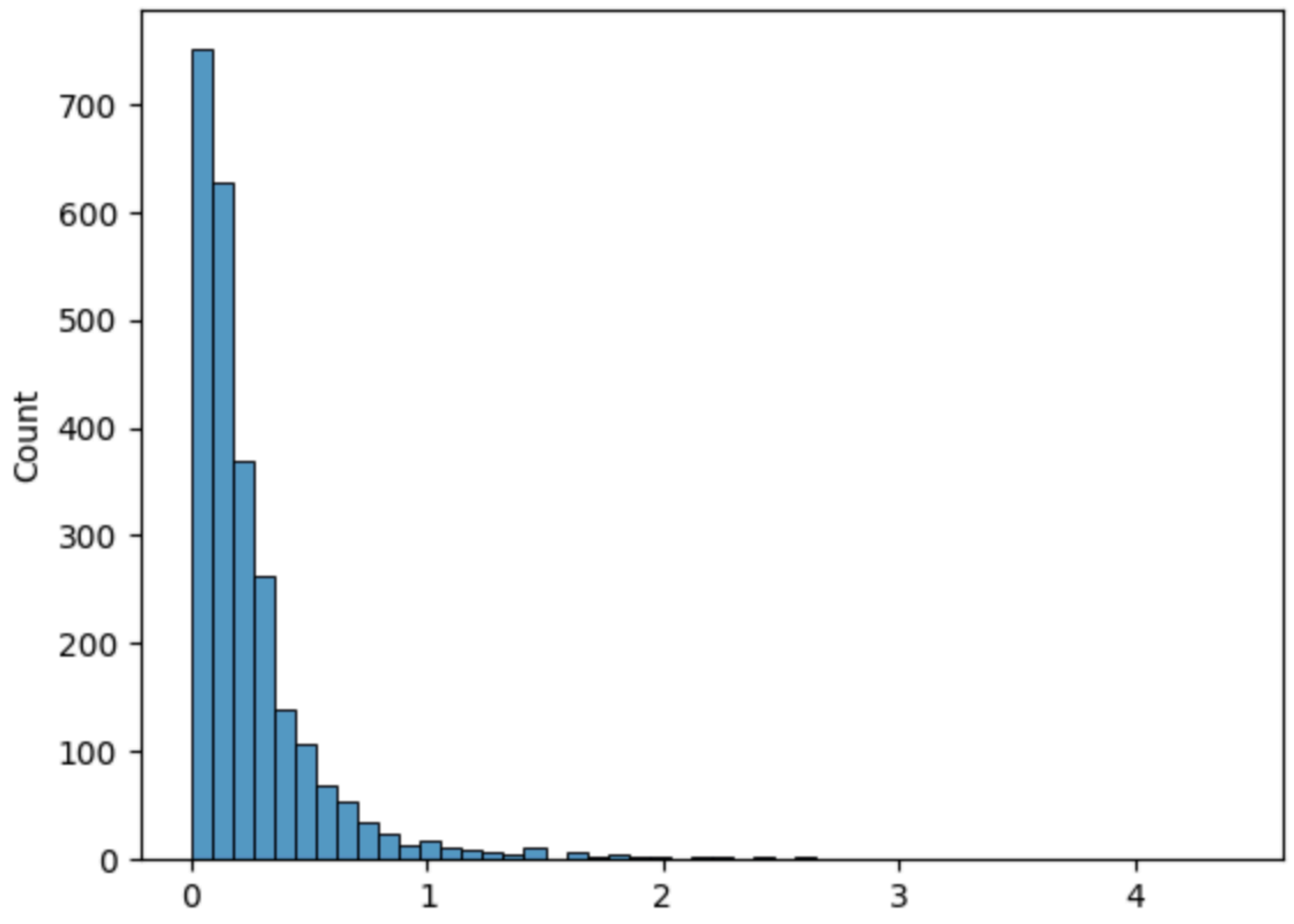
of the data points in that bin

PRACTICE OF DATA VISUALIZATION

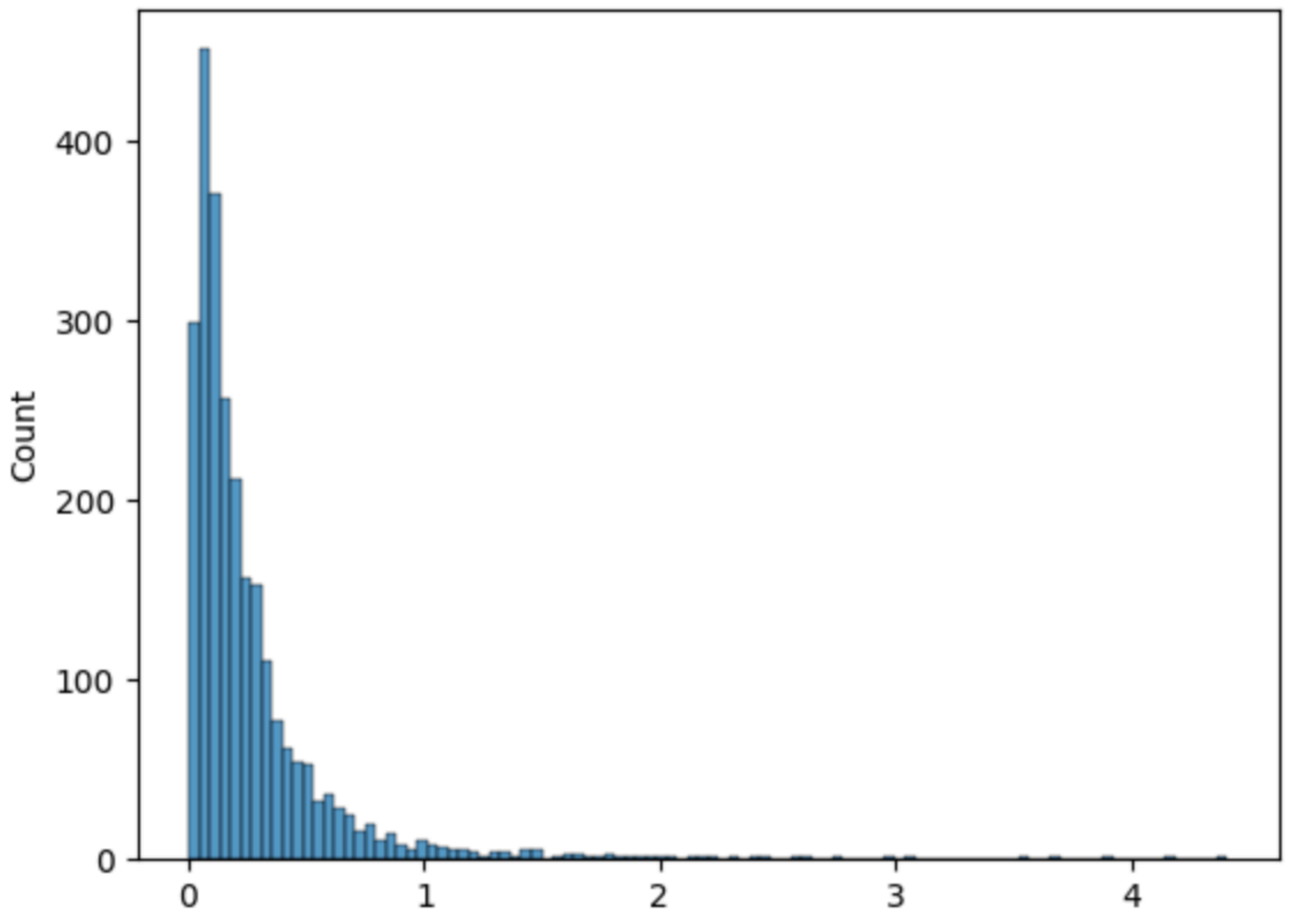
Impact of Bin size in a Histogram



10 bins



50 bins

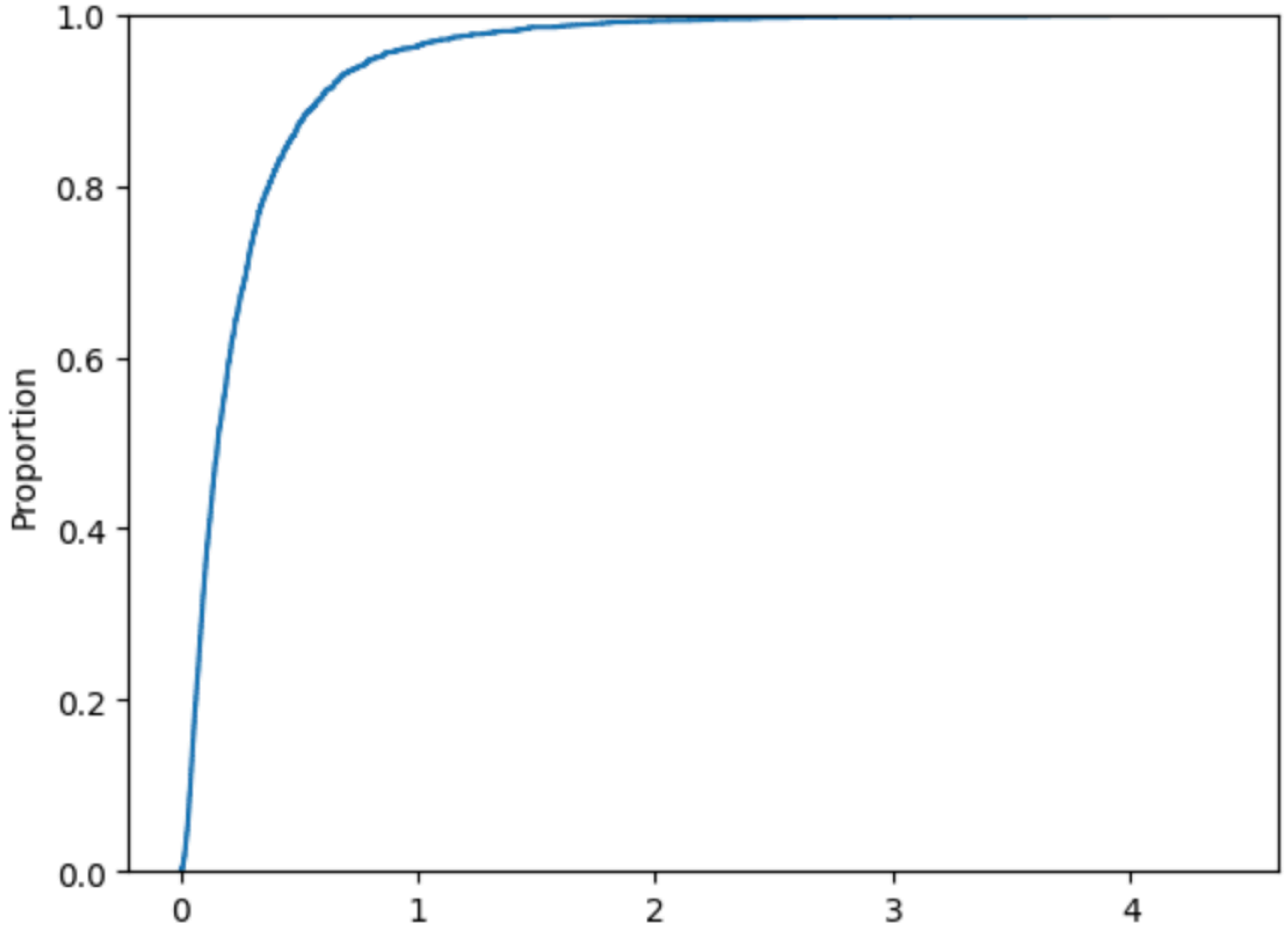


100 bins

PRACTICE OF DATA VISUALIZATION

ECDF: empirical cumulative distribution function

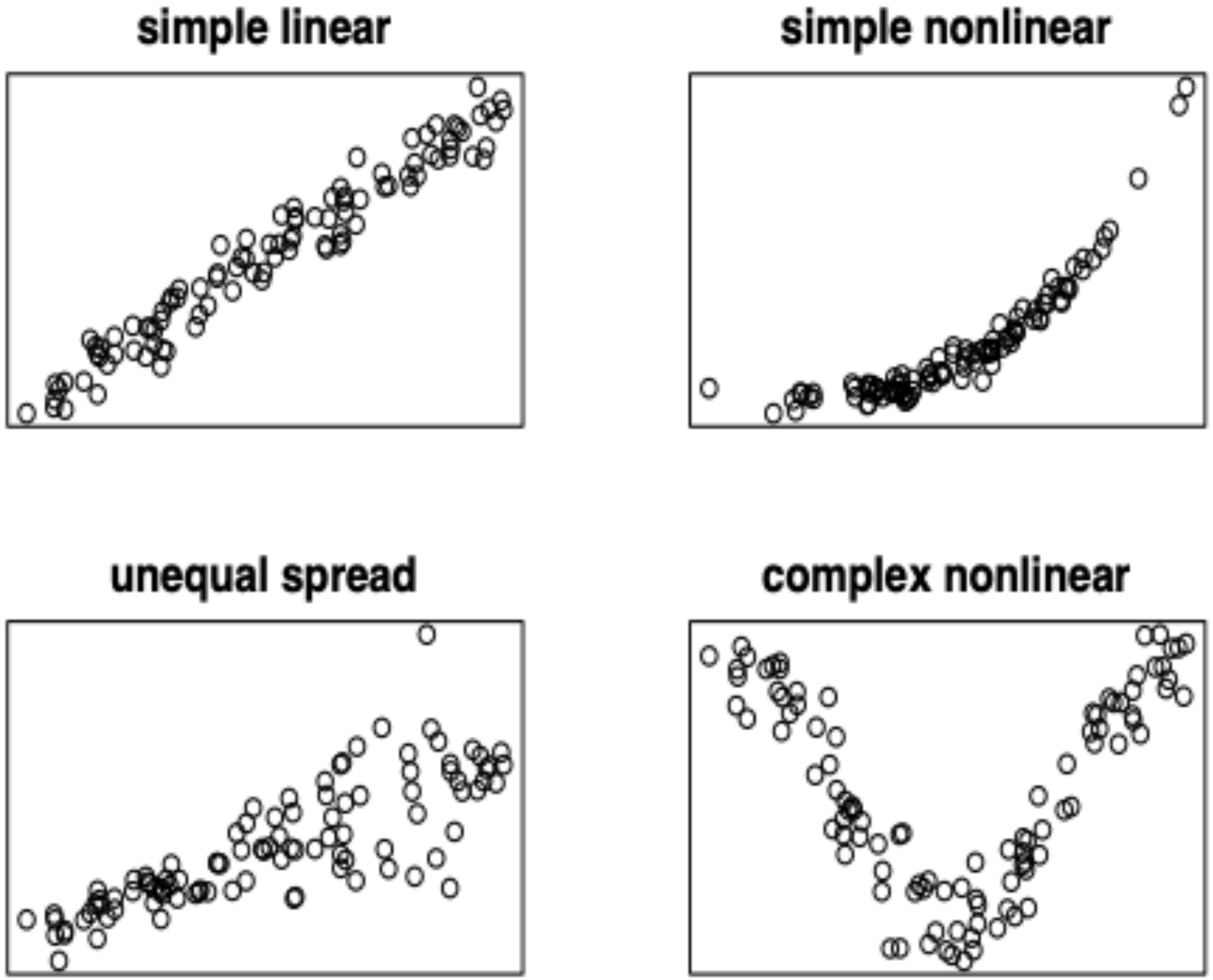
```
sns.ecdfplot(ted_main["comments"]/ted_main["duration"])
```



PRACTICE OF DATA VISUALIZATION

Visualizing Relationship Between Two Variables

Scatter plots



PRACTICE OF DATA VISUALIZATION

Visualizing Relationship Between Two Variables

Scatter plots

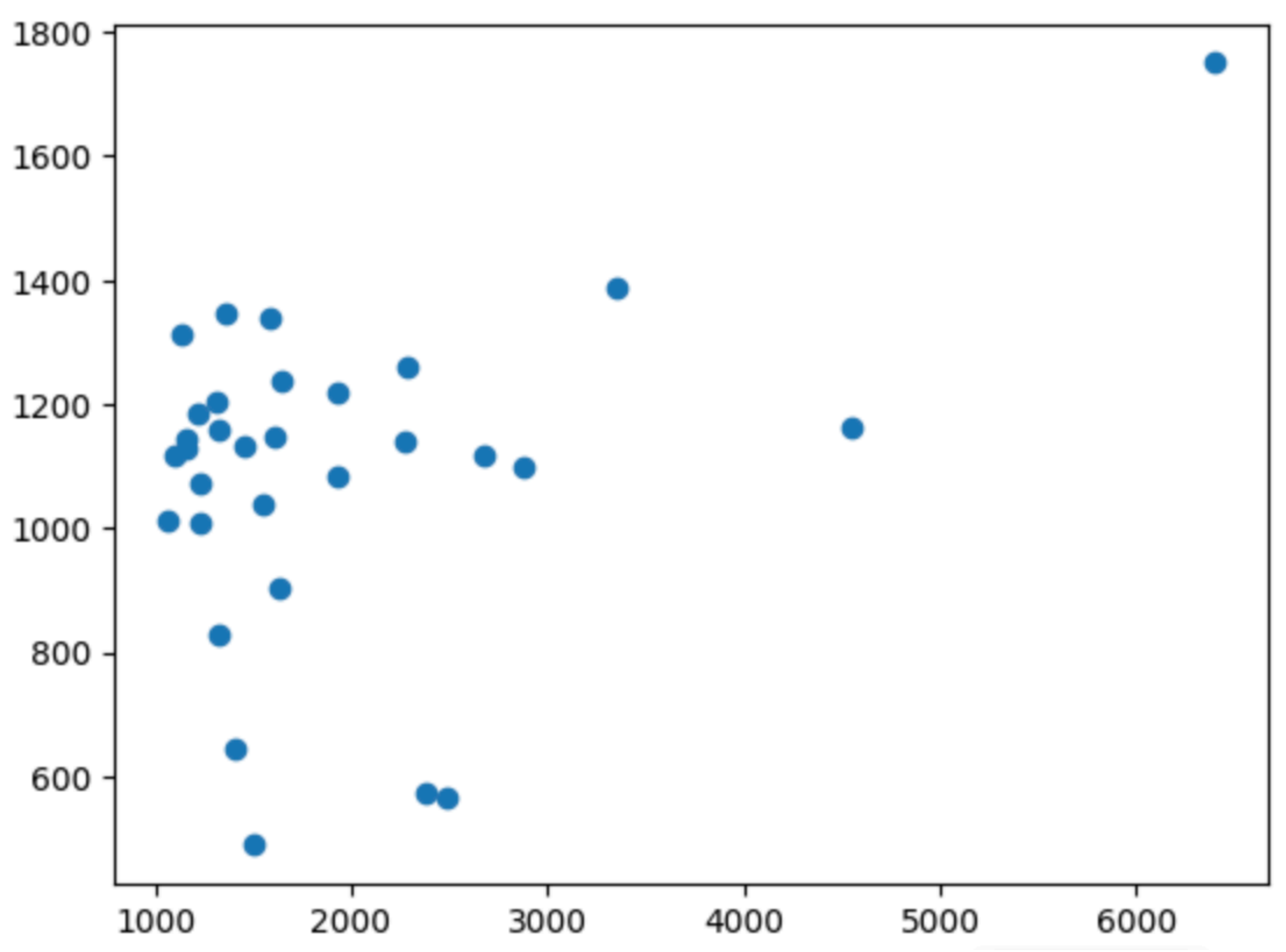
```
plt.scatter(ted_main['comments'][ted_main['comments']>1000],
            ted_main['duration'][ted_main['comments']>1000])
plt.show()
```

PRACTICE OF DATA VISUALIZATION

Visualizing Relationship Between Two Variables

Scatter plots

```
plt.scatter(ted_main['comments'][ted_main['comments']>1000],
            ted_main['duration'][ted_main['comments']>1000])
plt.show()
```



PRACTICE OF DATA VISUALIZATION

Visualizing Relationship Between Two Variables

Hex plots

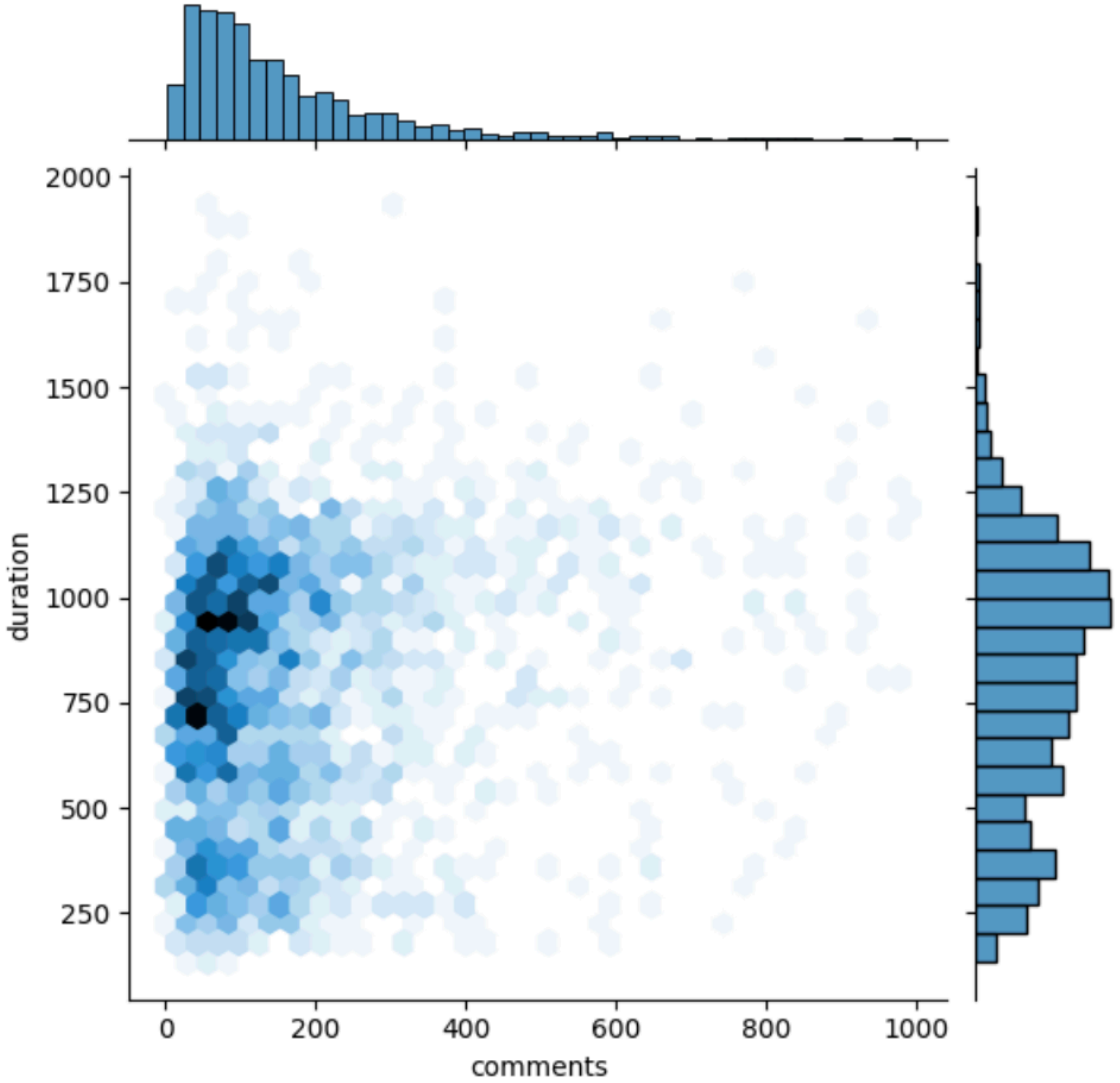
```
ted_filtered = ted_main[(ted_main['comments']<1000) &
                        (ted_main['duration']<2000)]
sns.jointplot(data=ted_filtered, x="comments", y="duration", kind="hex")
```

PRACTICE OF DATA VISUALIZATION

Visualizing Relationship Between Two Variables

Hex plots

```
ted_filtered = ted_main[(ted_main['comments']<1000) &
                        (ted_main['duration']<2000)]
sns.jointplot(data=ted_filtered, x="comments", y="duration", kind="hex")
```



Rather than individual datapoints, plot the density of their joint distribution.

- a two dimensional histogram.
- xy plane binned into hexagons.

Darker hexagons ⇒ more datapoints

PRACTICE OF DATA VISUALIZATION

Visualizing Relationship Between Two Variables

Contour plots

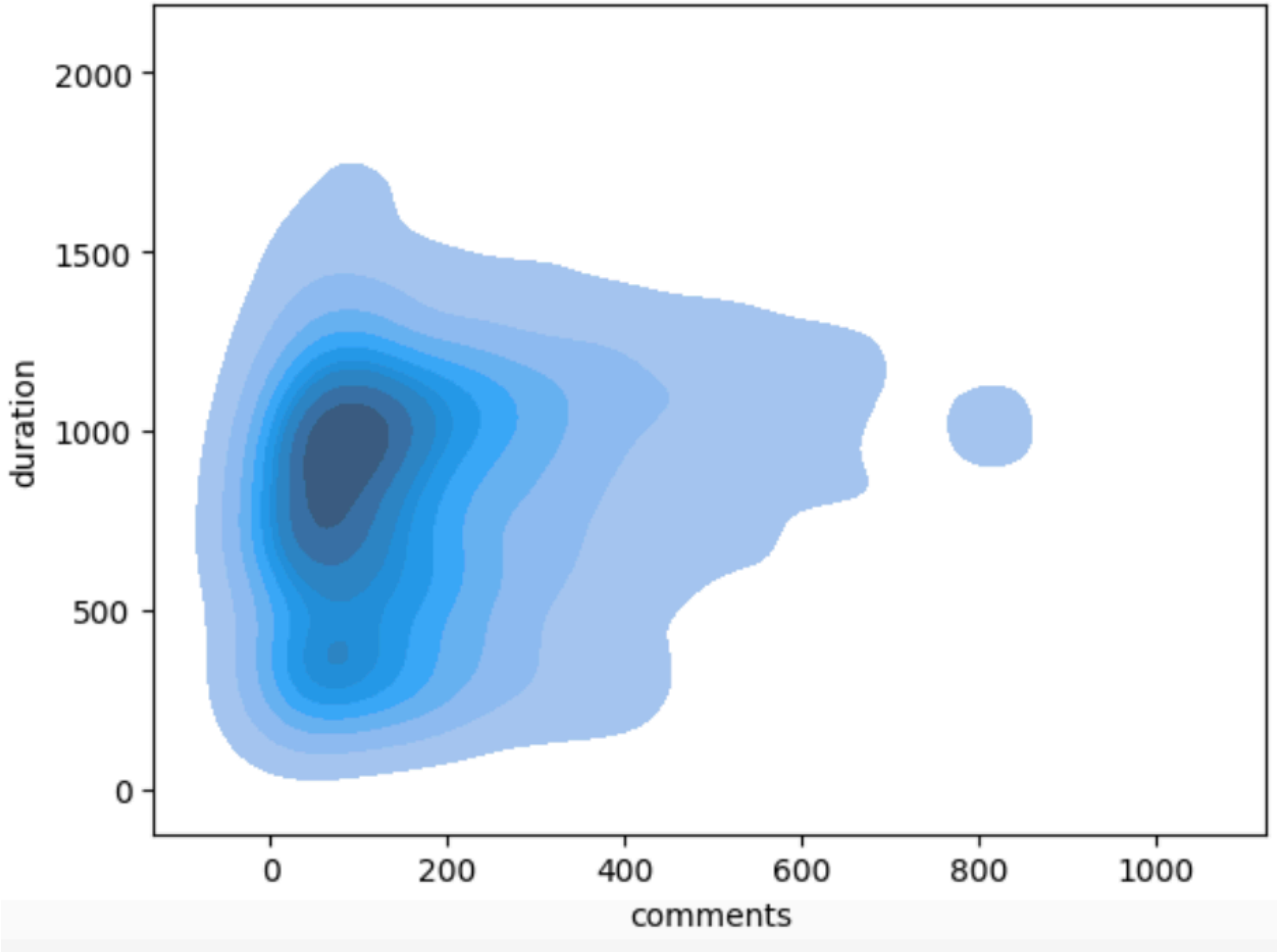
```
sns.kdeplot(data=ted_filtered,  x="comments", y="duration", fill=True)
```

PRACTICE OF DATA VISUALIZATION

Visualizing Relationship Between Two Variables

Contour plots

```
sns.kdeplot(data=ted_filtered,  x="comments", y="duration", fill=True)
```



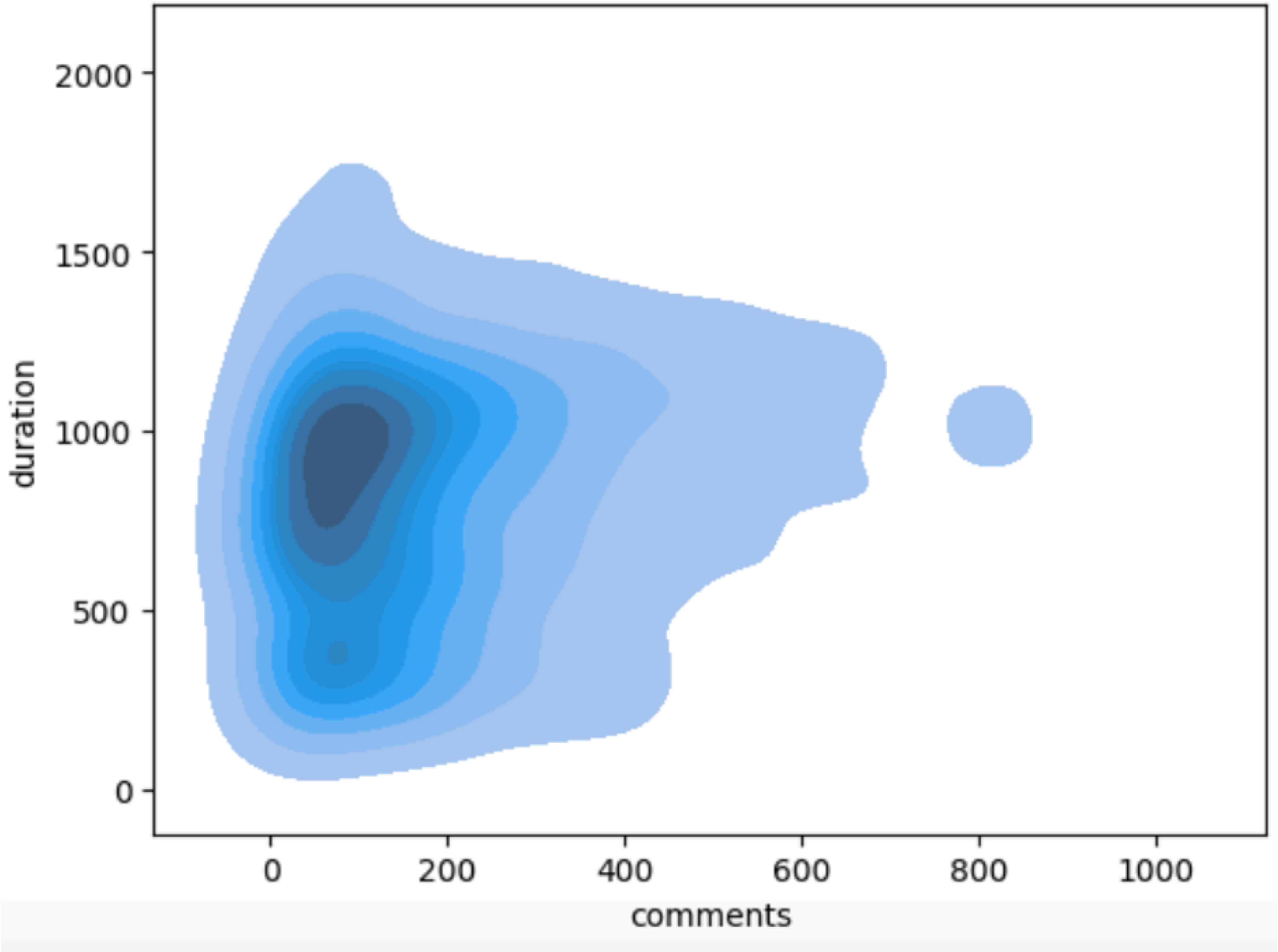
Contour lines represent an area with the same density of datapoints in it.

PRACTICE OF DATA VISUALIZATION

Visualizing Relationship Between Two Variables

Contour plots

```
sns.kdeplot(data=ted_filtered,  x="comments", y="duration", fill=True)
```



Contour lines represent an area with the same density of datapoints in it.

Compare against the scatter plot: